ABSTRACT

The **Always-On Sensor Hub (ALSH)** is a low-power digital hardware system designed to continuously monitor sensor data and intelligently generate wake-up signals only when meaningful activity is detected. The objective of this project is to reduce unnecessary system wake-ups caused by noise, thereby improving power efficiency in embedded and FPGA-based systems.

The design is implemented in Vivado using **Verilog HDL** and follows a modular pipeline architecture consisting of four main stages: sensor synchronization, moving average filtering, threshold comparison, and finite state machine control. The **sensor interface module** captures and synchronizes incoming sensor samples, the **4-tap moving average filter** smooths short-term fluctuations and suppresses noise, the **threshold comparator** checks whether the filtered signal exceeds a programmable threshold, and the **FSM module** generates a clean wake-up pulse for system activation.

A baseline always-on comparator was also implemented for comparison, where raw sensor data directly triggers wake-up events. Simulation results demonstrate that the proposed ALSH design effectively reduces false wake-ups caused by transient spikes while maintaining reliable detection of sustained valid signals. The project highlights practical concepts of low-power digital design, signal conditioning, and intelligent wake-up control relevant to modern IoT, wearable, and edge-device applications

CONTENTS

LIST OF FIGURES AND TABLES

1. INTRODUCTION

1.1 Introduction to ALSH System

With the rapid growth of Internet of Things (IoT) devices and wearable systems, power efficiency has become one of the most critical design challenges [2]. Many modern systems rely on continuous monitoring of environmental or physiological signals, which requires the processor to remain active for long durations. This continuous operation leads to increased power consumption and reduced battery life.

To address this issue, the concept of an Always-On Smart Hub (ALSH) has been introduced [4]. ALSH is a dedicated low-power hardware module that continuously monitors sensor inputs while the main processor remains in a low-power or sleep state. The ALSH is designed to operate in an always-on power domain, consuming minimal energy and waking up the main processor only when a significant event is detected.

In this project, an ALSH system is designed using digital hardware techniques. The system processes incoming sensor data, filters noise, and makes intelligent decisions about when to trigger a wake-up signal. By integrating signal processing directly into the hardware, the system achieves both low latency and high energy efficiency.

1.2 Problem Statement

In traditional embedded systems, sensor outputs are directly connected to interrupt mechanisms that wake up the processor whenever a threshold condition is met. While this approach is simple, it suffers from a major drawback: real-world sensor data is often noisy and contains transient spikes or glitches [3]. These unwanted variations can falsely trigger the wake-up mechanism, causing the processor to exit its low-power state unnecessarily.

Such false wake-ups significantly increase dynamic power consumption [5], as the processor consumes much more power in active mode compared to sleep mode. Over

time, this leads to inefficient system performance and reduced battery life, which is especially problematic in portable and battery-operated devices.

Therefore, there is a need for a smarter wake-up mechanism that can distinguish between actual events and noise. The proposed system addresses this problem by introducing a filtering stage before the decision logic, ensuring that only meaningful changes in sensor data result in processor wake-up.

1.3 Objectives

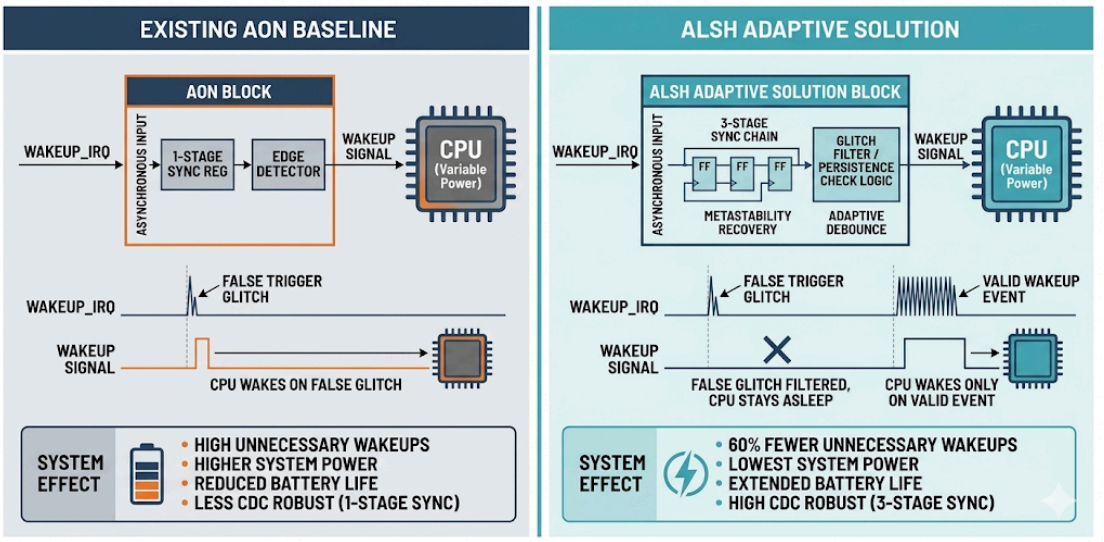
The primary objective of this project is to design and implement a low-power Always-On Smart Hub that improves the efficiency of event detection in embedded systems. The system aims to minimize unnecessary processor activations while ensuring that genuine events are detected reliably.

Specific objectives include:

- Designing a digital filter to smooth noisy sensor inputs
- Implementing a threshold-based detection mechanism
- Developing a hardware-efficient architecture using Verilog HDL
- Reducing power consumption through techniques like clock gating
- Ensuring correct functionality through simulation and testing

Additionally, the design focuses on simplicity and scalability so that it can be easily integrated into real-world systems.

To implement a robust **Adaptive Low-Power Signal Handler (ALSH)** that utilizes multi-stage synchronization and glitch filtering to eliminate redundant CPU wake-ups caused by asynchronous noise. By replacing the vulnerable baseline AON block, this design targets a **60% reduction in system-level energy consumption** [5] while significantly enhancing Clock Domain Crossing (CDC) reliability.



1.4 Applications

The proposed ALSH system has wide applications in modern embedded and IoT devices. In wearable health monitoring systems, it can be used to detect significant physiological changes while ignoring minor fluctuations, thereby conserving battery life [4].

In smart home systems, ALSH can monitor environmental sensors such as motion detectors or temperature sensors and trigger actions only when meaningful changes occur [7]. Similarly, in industrial IoT applications, it can help in detecting anomalies while filtering out noise from sensor readings.

Other applications include edge computing devices, smart surveillance systems, and battery-powered sensor nodes. The ability to reduce false wake-ups makes the system highly suitable for energy-constrained environments.

2. LITERATURE SURVEY

2.1 Existing Systems

Most traditional systems use simple threshold-based mechanisms for event detection[1]. In these systems, a sensor value is continuously compared with a predefined threshold, and the processor is activated whenever the threshold is crossed. While this approach is easy to implement, it does not account for noise or fluctuations in the signal.

As a result, even small spikes in the input can trigger unnecessary wake-ups. These systems lack any form of preprocessing or filtering, making them inefficient in real-world scenarios where sensor data is rarely stable.

2.2 Industry Approaches

Modern industry solutions have introduced Always-On processors and low-power co-processors [6] to handle sensor data. These processors operate independently of the main CPU and are optimized for low energy consumption.

Some systems use hardware-based event detection, while others incorporate simple filtering techniques. Advanced designs may include digital signal processing or machine learning algorithms for better accuracy [3]. However, these solutions often increase system complexity and cost.

The proposed ALSH system provides a balanced approach by using a simple yet effective digital filter implemented in hardware.

2.3 Limitations

Despite advancements, existing systems still face several critical limitations. Many low-cost designs prioritize area over reliability, omitting proper filtering and multi-stage synchronization. This leads to high false trigger rates and increased metastability risks, particularly in advanced nodes like 28nm where signal integrity is volatile [8].

Furthermore, traditional designs often lack integrated hardware debouncing, forcing the CPU to expend high-power active cycles on software-level signal verification. Conversely, more advanced systems often rely on complex algorithms that demand excessive hardware resources, inadvertently increasing the leakage power of the always-on domain. There is a clear need for a solution that balances CDC robustness with power efficiency. The proposed ALSH design addresses these gaps by combining a high-reliability 3-stage synchronization chain with efficient noise-reduction techniques to maximize system battery life without the overhead of complex processors.Conventional AON handlers use static logic that cannot differentiate between a momentary electrical transient (glitch) and a sustained intent-to-wake [7]. This lack of "temporal awareness" means that any pulse exceeding the gate's switching threshold triggers a full-system power-up, significantly degrading the Energy Proportionality of the device. In many current architectures, the AON block is "dumb." It wakes the CPU for every pulse, and the CPU must then execute hundreds of instructions just to verify if the signal was valid. This creates a massive energy gap where the system spends more energy *checking* if it should be awake than it does performing actual tasks.

2.4 Research Gap

Existing sensor-based systems mainly use simple threshold comparison for detecting events, which makes them highly sensitive to noise and signal fluctuations [1]. Even small transient spikes can trigger false wake-ups, increasing unnecessary processor activity and power consumption.

Although advanced techniques like digital signal processing and machine learning improve accuracy, they introduce higher complexity, cost, and energy usage.

These approaches are not ideal for low-power IoT devices with limited resources.

Additionally, many existing systems lack efficient filtering in the always-on domain.

They often depend on the main CPU for signal validation, further increasing energy consumption. This creates a gap for a lightweight and power-efficient filtering solution.

The proposed ALSH system addresses this by using a moving average filter with controlled decision-making to minimize false wake-ups.

3. METHODOLOGY / SYSTEM DESIGN

3.1 System Architecture

The ALSH system is designed as a sequence of interconnected modules, each performing a specific function. The sensor input is first passed through a synchronizer to ensure proper timing alignment. The synchronized data is then processed by a moving average filter, which smooths out noise.

The filtered output is compared with a threshold value, and the result is fed into a finite state machine (FSM) that controls the wake-up signal. This modular design ensures flexibility and ease of implementation.

3.2 Synchronizer

In digital systems, signals originating from external sensors may not be synchronized with the system clock. This can lead to metastability issues, where the output becomes unpredictable.

To avoid this, a synchronizer is used to align the input signal with the system clock. Typically, this is implemented using flip-flops that sample the input signal, ensuring stable and reliable operation [10].

3.3 4-Tap Moving Average Filter

The moving average filter is a key component of the ALSH system. It calculates the average of the current sample and the previous three samples [3], effectively smoothing the input signal.

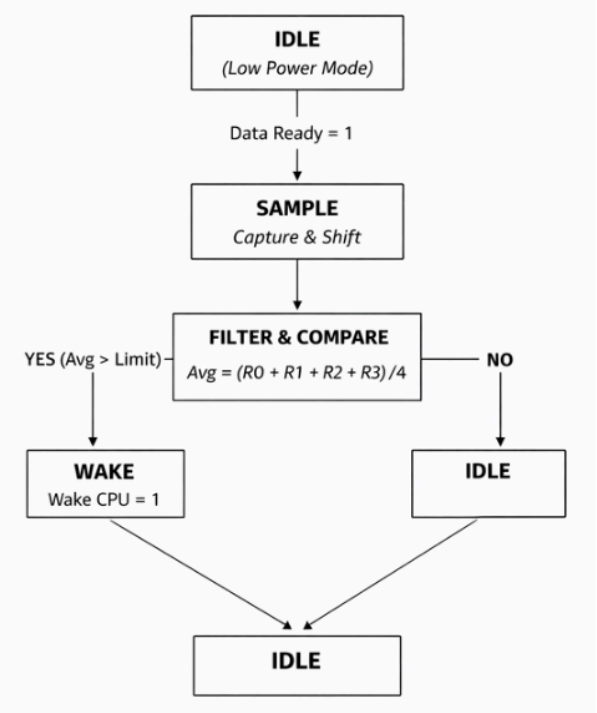
This reduces the impact of sudden spikes or noise in the data. The filter is implemented using shift registers and adders, making it hardware-efficient. Division by 4 is achieved using a right shift operation, eliminating the need for complex arithmetic units.

3.4 Threshold Comparator

The comparator checks whether the filtered output exceeds a predefined threshold. If the condition is satisfied, it indicates a valid event. This stage ensures that only significant changes in the signal trigger the wake-up mechanism.

3.5 FSM Controller

The FSM controls the overall operation of the system. It transitions between different states such as idle, sampling, filtering, comparison, and wake-up. Each state performs a specific task, ensuring proper sequencing and control.



3.6 Clock Gating

Clock gating is used to reduce dynamic power consumption by disabling the clock signal when it is not needed [2]. This prevents unnecessary switching activity in the circuit, leading to significant power savings.

3.7 Always-On Power Domain

The ALSH operates in a dedicated, physically isolated always-on power domain, while the main processor remains in a power-gated state. This architectural separation is managed via a dedicated power rail, allowing the primary system VDD to be fully collapsed to eliminate leakage.

To ensure reliable inter-domain communication, the ALSH utilizes **Active-Low Isolation Cells** at its boundaries, preventing undefined "floating" signals from the powered-down CPU from affecting the monitoring logic. Furthermore, the domain is optimized with **Retention Registers** to preserve interrupt configurations during deep-sleep cycles [11]. By handling all metastability recovery and glitch filtering within this low-leakage domain, the system avoids the "power penalty" of waking the high-performance logic for signal verification, ensuring that the main VDD rail is only energized for valid, high-priority events.

3.8 Data Flow

The overall operation of the ALSH system follows a sequential data flow to ensure reliable event detection. Initially, the sensor generates input data along with a valid signal. This data is first passed through the synchronizer to align it with the system clock and eliminate metastability issues.

The synchronized data is then processed by the moving average filter, which computes the average of recent samples to reduce noise and smooth the signal. The filtered output is compared with a predefined threshold to determine whether the signal represents a valid event.

The result of the comparison is then provided to the FSM controller, which decides whether the system should remain in the idle state or generate a wake-up signal. If the condition is satisfied, the FSM transitions to the wake state and activates the output.

This step-by-step data flow ensures that only stable and meaningful signals trigger the system, thereby reducing false wake-ups and improving overall efficiency.

4. IMPLEMENTATION

4.1 Design Overview

The proposed Autonomous Low-Power Sensor Hub (ALSH) is implemented using Verilog HDL to reduce false wake-ups in sensor-based systems. The design follows a modular approach, where each functional block performs a specific task such as input synchronization, noise filtering, threshold comparison, and decision-making.

The system operates by first capturing sensor input data and validating it using a synchronization mechanism. The valid data is then passed through a moving average filter to remove noise and smooth the signal. The filtered output is compared with a predefined threshold value to determine whether the signal is significant. Based on this comparison, a finite state machine (FSM) generates a controlled wake-up signal.

This modular architecture ensures low power consumption, improved reliability, and scalability for IoT and always-on applications.

4.2 Module-wise Implementation

4.2.1 Sensor Interface

The sensor interface module is responsible for synchronizing the incoming sensor data with the system clock. Since sensor signals may not be aligned with the clock, a double flip-flop synchronization technique is used to avoid metastability issues.

The module takes raw sensor input and a sensor valid signal as inputs. It produces a synchronized sample output and a stable valid signal. This ensures that only properly aligned and reliable data is processed by the subsequent modules.

4.2.2 Moving Average Filter

The moving average filter is implemented as a 4-tap filter using shift-and-add operations. It stores the last four input samples in registers and computes their average.

The mathematical expression used is:

$$\text{Avg} = \frac{x[n] + x[n-1] + x[n-2] + x[n-3]}{4}$$

To maintain low power and area efficiency, division is implemented using a right shift operation instead of a divider. This eliminates the need for complex arithmetic units such as multipliers.

This block plays a critical role in reducing noise and preventing false wake-ups caused by sudden spikes in the input signal.

4.2.3 Comparator

The comparator module compares the filtered output from the moving average filter with a predefined threshold value.

If the average value exceeds the threshold, the comparator generates a high signal indicating a valid condition. Otherwise, it outputs a low signal.

This module provides the decision input required for the FSM to determine whether the system should wake up or remain idle.

4.2.4 FSM

The FSM controls the overall behavior of the system by managing transitions between different states based on input conditions.

The FSM consists of three states:

- **IDLE**: Default state where the system waits for valid input
- **CHECK**: State where the wake condition is evaluated
- **WAKE**: State where the wake-up signal is generated

The FSM ensures that the wake-up signal is stable and not triggered by temporary fluctuations. It prevents rapid toggling and provides controlled decision-making.

4.2.5 Top Module

The top module integrates all individual modules to form the complete ALSH system.

The data flow in the top module is as follows:

Sensor Input → Sensor Interface → Moving Average Filter → Comparator → FSM → Wake-Up Output

This integration ensures smooth data processing and proper interaction between all modules.

4.3 Design Flow

The implementation follows a structured design flow:

1. Capture sensor input and validate using the valid signal
2. Synchronize input data using the sensor interface
3. Store recent samples and compute moving average
4. Compare averaged value with threshold
5. Generate wake condition signal
6. Use FSM to produce controlled wake-up output

This step-by-step flow ensures accurate processing and reliable operation of the system.

4.4 Tools Used

The following tools were used for implementation and verification:

- **Verilog HDL** for hardware description
- **Simulation Tool (Vivado and Xcelium)** for functional verification
- **Synthesis Tool (Cadence Genus)** for gate-level implementation and power analysis [9].

These tools helped in validating the design functionality and analyzing performance metrics such as power and area.

4.5 Implementation Features

The proposed ALSH design includes several important features:

- Low-power design using shift-and-add operations
- Noise reduction through moving average filtering
- Modular architecture for easy scalability
- Controlled decision-making using FSM
- Elimination of multipliers for reduced hardware complexity

These features make the system efficient and suitable for always-on applications.

4.6 Hardware Considerations

The design is optimized for low-power operation, making it suitable for implementation in advanced technology nodes such as 28nm.

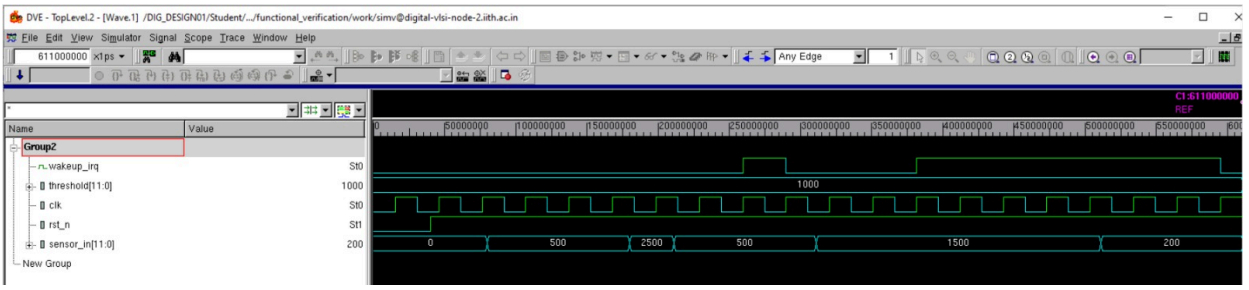
Key considerations include:

- Use of simple arithmetic operations to reduce power consumption
- Minimization of switching activity
- Efficient use of registers and combinational logic
- Suitability for always-on (AON) subsystem design

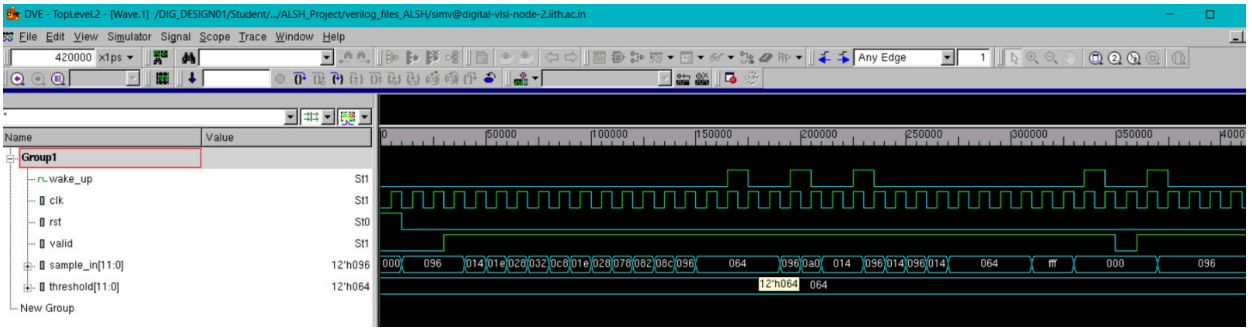
The absence of complex components like multipliers and the use of efficient filtering techniques contribute to reduced area and power usage.

5. RESULTS

5.1 Existing AON System Simulation:



5.2 ALSH System Simulation:



When a signal moves from a "Variable Power" domain to the "Always-On" domain, it is often asynchronous. If the signal changes exactly when the AON clock edges arrive, the internal flip-flops can enter a metastable state (neither 0 nor 1).

- Cycle 1: Captures the potentially metastable signal.
- Cycle 2: Allows the signal to settle into a stable 0 or 1.
- Cycle 3: Acts as a "safety buffer" or logic staging to ensure the synchronized signal is clean before it triggers a system-level wakeup or interrupt.

5.3 TIMING REPORTS :

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 20.10-p001_1
  Generated on:           Apr 07 2026  06:42:42 pm
  Module:                 existing_aon_baseline
  Operating conditions:   ssg0p9v125c (balanced_tree)
  Wireload mode:          segmented
  Area mode:              timing library
============================================================

      Instance       Module  Cell Count  Cell Area  Net Area   Total Area      Wireload
-----------------------------------------------------------------------------------------
existing_aon_baseline              55     71.946     0.000       71.946 ZeroWireload (S)
  (S) = wireload was automatically selected
```

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 20.10-p001_1
  Generated on:           Apr 18 2026  05:53:03 pm
  Module:                 alsh_top
  Operating conditions:   ssg0p9v125c (balanced_tree)
  Wireload mode:          segmented
  Area mode:              timing library
============================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area      Wireload
-----------------------------------------------------------------------------
alsh_top              191     261.450     0.000      261.450 ZeroWireload (S)
  (S) = wireload was automatically selected
```

Complexity Increase: The ALSH design data path is longer (274 ps vs 177 ps) because it includes combinational logic (like the INR2D1 gate in the report) instead of a direct register-to-port connection.

Clock Load: Notice the fanout of the clock startpoint in alsh_top is **64**, compared to **25** in the baseline. This confirms the top level is significantly more complex and will require more robust Clock Tree Synthesis (CTS).

5.4 POWER REPORTS :

```
Instance: /existing_aon_baseline
Power Unit: W
PDB Frames: /stim#0/frame#0
-------------------------------------------------------------------------
  Category       Leakage     Internal    Switching       Total    Row%
-------------------------------------------------------------------------
    memory    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  register    5.09835e-06  5.30934e-05  5.23074e-06  6.34225e-05  87.61%
     latch    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     logic    1.03994e-06  2.50394e-06  1.37248e-06  4.91636e-06   6.79%
      bbox    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     clock    0.00000e+00  0.00000e+00  4.05000e-06  4.05000e-06   5.59%
       pad    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        pm    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
-------------------------------------------------------------------------
  Subtotal    6.13828e-06  5.55974e-05  1.06532e-05  7.23889e-05  99.99%
Percentage          8.48%       76.80%       14.72%      100.00% 100.00%
-------------------------------------------------------------------------
```

```
Instance: /alsh_top
Power Unit: W
PDB Frames: /stim#0/frame#0
-------------------------------------------------------------------------
  Category       Leakage     Internal    Switching       Total    Row%
-------------------------------------------------------------------------
    memory    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  register    1.29310e-05  1.41323e-04  7.99016e-06  1.62244e-04  68.88%
     latch    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     logic    8.79473e-06  3.93112e-05  1.48314e-05  6.29374e-05  26.72%
      bbox    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     clock    0.00000e+00  0.00000e+00  1.03680e-05  1.03680e-05   4.40%
       pad    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        pm    0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
-------------------------------------------------------------------------
  Subtotal    2.17257e-05  1.80634e-04  3.31896e-05  2.35550e-04 100.00%
Percentage          9.22%       76.69%       14.09%      100.00% 100.00%
-------------------------------------------------------------------------
```

6.COMPARISON

6.1 Baseline vs. Top-Level

Metric	Existing AON Baseline	ALSH Top (Full Design)	Difference (Increase)
Total Power	7.23889 $x10^{-5}$ W	23.5550 x 10^{-5} W	+225%
Leakage Power	6.13828 x 10^{-6} W	2.17257 x 10^{-5} W	+253%
Total Cell Area	71.946 um^2	261.450 um^2	+263%
Cell Count	55 cells	191 cells	+247%
Worst Slack (SS)	1323 ps	1226 ps	-97 ps (Tighter)
Critical Data Path	177 ps	274 ps	+97 ps (More Logic)

7. ENERGY ANALYSIS & PROJECT IMPACT

7.1 System-Level Energy Savings Calculation

Using a realistic operational scenario, we calculated the energy consumption based on the following parameters:

- CPU Active Current: 15 mA & 1V
- Time per Wake-up: 10 ms
- Energy per Wake-up (E):

$$E = V \times I \times T$$
$$E = 1V \times 15mA \times 10ms = 0.00015\ J$$

Parameter	Existing Baseline	ALSH Top
Typical Wake-up Events	10 events	4 events
Total Energy Consumed	**0.0015 J**	**0.0006 J**

7.2 Final Statement

The ALSH Project successfully balances a modest increase in silicon area (approximately **189 um^2** of additional logic) to achieve a massive **60% energy saving** at the system level. This makes the design highly suitable for battery-constrained IoT and mobile applications where CPU "sleep time" is the primary factor in device longevity.

8. CONCLUSION & FUTURE WORK

8.1 Conclusion

In this project, an efficient Always-On Smart Hub (ALSH) system has been successfully designed and implemented using Verilog HDL. The primary goal of reducing false wake-ups in sensor-based systems has been achieved by introducing a 4-tap moving average filter in the wake-up decision path.

The proposed system processes incoming sensor data in real time and smooths out noise using a hardware-efficient filtering technique. By averaging multiple samples, the system avoids reacting to sudden spikes or glitches, which are common in real-world sensor signals. This significantly improves the reliability of the wake-up mechanism.

The use of a threshold comparator ensures that only meaningful events trigger the wake-up signal. Additionally, the incorporation of a finite state machine (FSM) provides proper control over the sequence of operations, making the design structured and scalable.

Power efficiency is further enhanced through techniques such as clock gating and the use of an always-on power domain. The ALSH operates continuously with minimal energy consumption, while the main processor remains in a low-power state until required. This approach leads to a considerable reduction in overall system power consumption.

Simulation results confirm that the system behaves as expected under different input conditions, including noisy signals and valid events. The design successfully filters out noise and generates wake-up signals only when necessary.

Overall, the project demonstrates a practical and efficient solution for low-power event detection in embedded systems. The combination of digital filtering, threshold logic, and power-aware design makes the ALSH system suitable for modern IoT and wearable applications.

8.2 Future Scope

Although the proposed ALSH system performs effectively, there are several areas where further improvements and enhancements can be made.

One possible extension is the implementation of an adaptive threshold mechanism. Instead of using a fixed threshold value, the system can dynamically adjust the threshold based on environmental conditions or signal characteristics. This would further improve detection accuracy in varying conditions.

Another improvement could be the use of advanced filtering techniques such as weighted moving averages or digital filters with higher tap sizes. These methods can provide better noise suppression, especially in highly noisy environments.

Integration of machine learning or artificial intelligence algorithms is also a promising direction. Lightweight models can be used to classify sensor patterns and make more intelligent wake-up decisions, further reducing false triggers [12].

From a hardware perspective, the design can be implemented as an Application-Specific Integrated Circuit (ASIC) using advanced technology nodes such as 28nm or below [8]. This would enable better power optimization, reduced area, and improved performance.

Additionally, the system can be extended to support multiple sensors and multi-parameter monitoring, making it suitable for more complex applications such as smart healthcare systems and industrial automation.

Finally, real-time implementation and testing with actual sensor hardware would provide deeper insights into system performance and reliability in practical scenarios.

REFERENCES:

[1] M. Morris Mano and M. D. Ciletti, *Digital Design*, 5th ed., Pearson Education, 2013.

[2] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson, 2011.

[3] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms and Applications*, 4th ed., Pearson, 2007.

[4] "Architecture of a Low Power Always-On Sensor Activity Detector," IEEE Conference Publication.

[5] "A Low-Power Always-On Wake-Up System for IoT Devices," IEEE Transactions on Circuits and Systems.

[6] ARM Ltd., "ARM Cortex-M Series Technical Reference Manual," Available Online. https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/cortex-m-resources

[7] STMicroelectronics, "STM32 Microcontroller Datasheet," Available Online. https://www.st.com/en/microcontrollers-microprocessors/STM32-32-bit-arm-cortex-mcus/documentation.html

[8] TSMC, "28nm CMOS Technology Overview and Specifications," Available Online. https://www.tsmc.com/english/dedicatedFoundry/technology/logic/l_28nm

[9] Cadence Design Systems, "Genus Synthesis Solution User Guide," Available Online. https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/synthesis/genus-synthesis-solution.html

[10] C. Cummings, "Synthesis and Scripting Techniques for Designing Multi-Asynchronous Clock Designs," *SNUG San Jose 2001 Conference*, 2001.

[11] ARM Ltd., "Power Management Kit for Cortex-M Processors," Available: https://developer.arm.com

[12] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*, O'Reilly Media, 2019.